

CS 333 : Compiler Design TUTORIAL on Static Semantics

P1. Consider the following grammar that generates infix expressions using the terminals { **addop**, **num** }, where **addop** denotes the binary operators { +, - } and **num** is an integer.

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow \text{addop } T R_1 \mid \epsilon \\ T &\rightarrow \text{num} \end{aligned}$$

For instance, the infix expression, 25 + 30 – 15 – 40 + 10, can be generated by this grammar. You have to write a SDTS whose purpose is to convert this infix expression to, 25 30 + 15 – 40 – 10 +, which is its equivalent postfix expression. You may assume that lexical analyser returns the operator as **addop.lexeme** and the value of **num** as **num.val**; you are permitted to change the grammar.

P2. Write a translation scheme for converting postfix expressions to infix over the alphabet {addop, num} on similar lines as P1. Your SDTS should work for both top down and bottom-up parsers, if a single SDTS does not serve write two separate ones for the purpose.

P3. As a preparation towards SDTS for processing declarations in C language the following information is provided. The **basic types** to be supported are **int** and **float**, while the **derived types** are **pointers** and **multi-dimensional arrays**. The attributes of interest are **identifier name** (scalar or array), its **type**, its **width** (number of bytes) and **number of dimensions** (0 for scalar and m for a m-dimensional array declaration). The name and its attributes are inserted into a symbol table, if the name does not exist in the table, else an error is reported indicating redeclaration of the same name. The **type expression** attribute is to be constructed as a string along with parsing; **width** attribute is an integer that gives the space, in bytes, required to store the name, the **dimension** attribute is 0 for **scalars** and n for an **n-dimensional array**. The **offset** attribute is **calculated for each name and gives its** relative position in the symbol table. The desired symbol table structure is given below. The possible values of scope are {global / local / parameter}.

```
int a, *p;
int *q[10][20];
float b, x[100], y[10][50];
```

Index	Name	Type expression (name)	scope	Width in bytes	Dimensions	Offset
0	a	int		4	0	0
1	p	pointer(int)		8	0	4
2	q	array(10,array(20, pointer(int)))		1600	2	12
3	b	float		4	0	1612
4	x	array(100, float)		400	1	1616
5	y	array(10, array(50, float))		2000	2	2016

Declarations	Exercise
int a; short b; float x[16]; double y[20]; void f(int z)	Draw the symbol table(s) that should be constructed after processing the declarations given above. Also write a CFG for generating program fragments of the kind given alongside.

P4. Write a SDTS for handling **struct** and **union** declarations in C, including **nested structs**. The built-in types are int, char and derived types are arrays and pointers. The SDTS should address the important issues such as symbol table organization and semantic errors. Outline your objective in simple english first and then provide the SDTS.

P5. Do the assignment of P3 but add **const** objects of the types mentioned therein. Identify the requirements to handle const objects and semantic errors they could be involved in. Outline your objective in simple english first and then provide the SDTS.

P5. Do the same exercise as for P4 for the program fragment given below.

```
struct rec{
    int data;
    struct rec* next;
};
int g = 100;
void f( int i, int *p, int& j)
{static int cnt = 0;
  rec x;
  rec* y;
  y = (rec*) malloc(sizeof(rec));
  x.next = y; x.data = j + *p; cnt+= i;
  return;
}
```

P6. We would like to process program fragments, as given below for compilation and intermediate code generation.

```
integer a [10, 20];
integer b [10, 20];
integer i, j;
i = 10;
j = 16;
b[i, j] = a [j, i] + a [j , j] + 25;
```

Make appropriate changes to the Translation Scheme given below for array references so that code as given above, last 3 lines only, can be handled along with parsing. Grammar and semantic rules for processing of declarations (scalars and arrays) are not required. Arrays are to be assumed to be in row-major representation and their attributes may be accessed from the symbol table. The grammar that you write should be able to produce strings of the form given in the last 3 lines of the code above.

```
S → L := E { if L.offset = null then emit (L.place ':=' E.place)
              else emit (L.place '[' L.offset ']' ':=' E.place) }
E → L { if L.offset = null then E.place ':=' L.place
        else { E.place := newtemp(); emit (E.place ':=' L.place '[' L.offset ']) } }
L → elist ] { L.place := newtemp(); L.offset := newtemp(); emit (L.place ':=' c(elist.array));
              emit (L.offset '=' elist.place '*' width (elist.array)) }
L → id { L.place := id.place; L.offset := null }
elist → elist1 , E { t := newtemp(); m := elist1.dim + 1; emit (t ':=' elist1.place '*' limit (elist1.array, m);
                    emit (t ':=' t '+' E.place); elist.array := elist1.array; elist.place := t; elist.dim := m }
elist → id [ E { elist.array := id.place; elist.place := E.place; elist.dim := 1 }
```

Recall that for the n dimensional array reference $\mathbf{a}[u_1, u_2, \dots, u_n]$, where u_i 's are the upper bounds and the lower bounds are assumed to be 1, the relative address for the reference $\mathbf{a}[e_1, e_2, \dots, e_n]$ is given by $c + v$, where the expressions of c and v are as given below :

$$c = \text{addr } \mathbf{a}[1, 1, \dots, 1] - ((\dots ((u_2 + 1) * u_3 + 1) \dots u_n + 1) * w$$

$$v = ((\dots ((e_1 * u_2 + e_2) * u_3 + e_3) * \dots) * u_n + e_n) * w$$

The attribute `elist.array` is a pointer to the symbol table (ST) for the array name, `elist.place` is a placeholder for the v part of the address calculation, `elist.dim` keeps track of the dimension information, `c(array_name)` returns the c part of the address formula from the ST, `width(array_name)` gives the width information from ST and `limit(array_name, m)` returns the upper bound for the dimension m from the symbol table.

Show the intermediate code that should be generated for the last 3 lines for the program fragment.

P7. For the same context as of P6, except that arrays are now assumed to be in column-major representation, change the grammar and/or SDTS as appropriate. Formulate recurrence relations for the constant and variable part for this situation. The grammar that you write should be able to produce strings of the form given in the last 3 lines of the code above and also generate correct intermediate code.

P8. For the same context as P6 above, change the grammar rules, if so needed, so that assignment expressions of the form shown below can be processed for parsing.

$\mathbf{b[i+2, i+j]} = \mathbf{a[i+j-2, 2*i]} + \mathbf{a[j+i, i+j-10]} + 25;$

Assume that a separate terminal, **num**, is given by the lexer when an integer constant is recognized. The only operators permitted in expressions are addition (+) and multiplication (*). Now introduce semantic actions along with your grammar rules so that intermediate code is generated for the assignment expressions of part (a) above. Show the intermediate code generated for the given assignment.

P9. We would like to process program fragments, as given below for compilation and intermediate code generation.

```
integer a [10] [20];
integer b [10] [20];
integer i;
integer j;
i = 10; j = 16;
b[i][j+2] = a[i-1][j+1] + a[i+1][j-2];
```

Make suitable changes to the SDTS given above so that processing of array references and assignments as given in the last 3 lines of the program fragment can be handled including generation of intermediate code.

P10. Using the same context as of P9 above, the problem here is to process array declarations of the form shown in the program fragment in the first two statements only so that the references of the form shown in the last statement can be processed for parsing. Introduce grammar rules with semantic actions so that array declarations of the kind given in the first two statements can be processed. The semantic analysis should store relevant information in the Symbol Table for the array declaration, in particular the **c part** of the address calculation should be carried out and kept along with other entries for the array in the ST. Show the symbol table entries after processing of these two declarations.

```
integer a [10, 20];
integer b [10, 20];
```

P11. Formulate recurrence relations for c and v for the array reference $\mathbf{a}[e_1, e_2, \dots, e_n]$, such that a) row major representation is used, and b) array declaration is of the form : $\mathbf{a}[l_1: u_1, l_2: u_2, \dots, l_n: u_n]$, where the lower and upper bounds, l_i and u_i may be negative, 0 or positive and $l_i \leq u_i, \forall i$. An array declaration such as $\mathbf{A[-25:-2][3:20]}$ is permitted in this setting.

P12. Consider the two SDTS given for evaluation of a boolean expression using complete evaluation and partial evaluation (given in the Appendix). For the input string,

$a + b > c + d$ and $a * a \geq 20$ or not ($a == b$ or false)

- (a) generate the intermediate code produced using both the SDTS,
- (b) compare the codes in terms of various attributes such as length, the number of incomplete targets, etc.

P13. Consider the grammar for translation of control flow constructs which uses partial evaluation of boolean expressions. Is the last semantic action, **emit ('goto' $M_1.stat$)**, in the rule for the **while – loop** absolutely necessary? If your answer is no, rewrite the semantic action for this rule, without changing those for the other rules. Otherwise, produce an input (as small as possible) such that removing the last action leads to generation of erroneous intermediate code.

P14. Apply the SDTS of control flow constructs (given in Appendix) to the following input and show how the intermediate code generation along with evaluation of semantic attributes takes place step by step.

```
while a+b > c + d do
begin
  if a > b then if b > c then begin d := b + c; b := b / 2 end
                else while a > 0 do a := a - 1;
                else while b > 0 begin sum := sum + b; b := b - 1 end
end;
```

P15. We would like to add the C loop construct, **do S while B** to the control flow constructs grammar. The semantics of this statement is that the body S is executed at least once. Subsequent execution of the body S depends on the boolean expression B, in case B evaluates to false, the loop is exited.

$S \rightarrow \text{do } S_1 \text{ while } B$

Write a SDTS first and then apply it on the following input and show how the intermediate code generation along with evaluation of semantic attributes place step by step.

```
do
begin
  a := c + d;
  if ( a < 0 ) then c := c*d;
  else
    begin c := c + d; while ( c > 0) do c := d / c; end
  end
while ( a <= b)
```

P16. Add a rule for the **for loop** construct to the control flow constructs grammar. Use the C language syntax and semantics. Write the required semantic analysis first in English. Convert to SDTS ; construct a suitable input string and show the intermediate code generation along with evaluation of semantic attributes using your SDTS.

P17. It is intended to add **break statement** as an instance of S to the control flow constructs grammar. The **break** statement terminates the execution of the nearest enclosing loop or conditional statement in which it appears. Control passes to the statement that follows the terminated statement, if any. It is typically used in the switch and loop statements.

$S \rightarrow \text{break}$

(a) Rewrite the grammar given to include the break statement, as described above, by making minimal changes. This part concerns with syntax only and semantic issues may be ignored.

(b) Explain your scheme for semantic action for the break statement and implication of this statement on others in English, justifying the same.

(c) Convert the semantic actions of part (b) above to SDTS to generate intermediate code, using synthesized attributes and globals only. You have to write only a) the actions for the new rules, and (b) new actions for the existing rules, if so required

(d) Use your translation scheme to generate intermediate code for the following input; given that a,b,c and d are integers.

```
while ( a <= b)
begin
  a := c + d;
  if ( a < 0 ) then break
  else
    begin c := c + d; if ( c > 0 ) then d := d / c else break end
end;
```

P18. It is intended to add **continue** as an instance of S to the control flow constructs grammar. The **continue** statement terminates the execution of the current iteration in the nearest enclosing loop in which it appears and continues with the next iteration, if any. It is typically used in loop statements. Causes the remaining portion of the enclosing loop, such as a for, while or do-while loop body to be skipped. The continue statement causes a jump to the end of the loop body.

$S \rightarrow \text{continue}$

(a) Rewrite the grammar given to include the continue statement, as described above, by making minimal changes. This part concerns with syntax only and semantic issues may be ignored.

(b) Explain your scheme for semantic action for the continue statement and implication of this statement on others in English, justifying the same.

(c) Convert the semantic actions of part (b) above to SDTS to generate intermediate code, using synthesized attributes and globals only. You have to write only a) the actions for the new rules, and (b) new actions for the existing rules, if so required

(d) Use your translation scheme to generate intermediate code for the following input; given that a,b,c and d are integers.

```
while ( a <= b)
begin
  a := c + d;
  if ( a == 0 ) then continue;
  if ( a < 0 ) then break
  else
    begin
      c := c + d;
      if ( c > 0 ) then d := d / c else continue ;
    end
end;
```

APPENDIX – 1

SDTS FOR VARIOUS LANGUAGE FEATURES

Infix to Postfix

$E \rightarrow T R$
 $R \rightarrow \text{addop } T \{ \text{print}(\text{addop.lexeme}) \} R_1 \mid \epsilon$
 $T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$

Declaration Processing

$P \rightarrow D$
 $D \rightarrow D; D$
 $D \rightarrow T L; \quad \{ \text{enter} (L.\text{list}, T.\text{type}, T.\text{width}, \text{offset})$
 $L \rightarrow L_1, \text{id} \quad \{ L.\text{list} := \text{append}(L_1.\text{list}, \text{id.name}) \}$
 $L \rightarrow \text{id} \quad \{ L.\text{list} = \text{makelist} (\text{id.name}) \}$
 $T \rightarrow \text{integer} \quad \{ T.\text{type} := \text{integer}; T.\text{width} := 4 \}$
 $T \rightarrow \text{real} \quad \{ T.\text{type} := \text{real}; T.\text{width} := 8 \}$
 $T \rightarrow \text{id} [\text{num}] \text{ of } T_1 \{ T.\text{width} := \text{num.val} * T_1.\text{width};$
 $\quad \quad \quad T.\text{type} := \text{array} (\text{num.val}, T_1.\text{type}) \}$
 $T \rightarrow \uparrow T_1 \quad \{ T.\text{type} := \text{pointer} (T_1.\text{type}); T.\text{width} := 4 \}$

Alternate SDTS

$D \rightarrow D; D$
 $D \rightarrow D_1, \text{id} \quad \{ \text{enter} (\text{id.name}, D_1.\text{syn}); D.\text{syn} := D_1.\text{syn} \}$
 $D \rightarrow T \text{ id} \quad \{ \text{enter} (\text{id.name}, T.\text{type}); D.\text{syn} := T.\text{type} \}$
 $T \rightarrow \text{integer} \mid \text{real} \mid \dots \quad \{ \text{same as earlier} \}$

SDTS for Procedure Declarations

We use the earlier grammar for illustration but add another rule to it.

$P \rightarrow D$
 $D \rightarrow D; D$
 $D \rightarrow \text{id} : T$
 $D \rightarrow \text{proc id} ; D ; S$
 $T \rightarrow \text{integer}$
 $T \rightarrow \text{real}$
 $T \rightarrow \text{array} [\text{num}] \text{ of } T_1$
 $T \rightarrow \uparrow T_1$

The rule, $D \rightarrow \text{proc id} ; D ; S$ permits nested procedures, but without parameters. The design solution is to create a separate table for handling names local to each procedure. The tables are linked in such a manner that lexical scoping rules are honoured.

SCOPE INFORMATION

For symbol table management, the following organization and routines are assumed.

1. Each table is an array of a suitable size which is linked using the fields as shown.
2. There are 2 stacks, called **tableptr** and **offset**. The stack tableptr points to the currently active procedure's table and the pointers to the tables of the enclosing procedures are kept below in the stack.
3. The other stack is used to compute the relative addresses for locals within a procedure.
4. **mktable(previous)** is a procedure that creates a new table and returns a pointer to it. It also gives the pointer to the previous table through the argument, **previous**. This is required in order to link the new table with that of the closest enclosing procedure's.

5. **enter**(table, name, type, width, offset) is a procedure for creating a new entry for the local variable, name, along with its attributes.
6. **enter**(table, name, type, width, offset) is a procedure for creating a new entry for the local variable, name, along with its attributes.
7. **addwidth**(table,width) is used to compute the total space requirement for all the names of a procedure.
8. **enterproc** (table, name, nest level, newtable) creates a new entry for a procedure name. The last argument is a pointer to the corresponding table.

The semantic rules are given now.

$P \rightarrow M D \quad \{ \text{addwidth}(\text{top}(\text{tableptr}), \text{top}(\text{offset})) ; \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset}) \}$
 $M \rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{nil}); \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}); \text{nest} := 1 \}$
 $D \rightarrow D ; D$
 $D \rightarrow \text{id} : T \quad \{ \text{enter}(\text{top}(\text{tableptr}), \text{id.name}, T.\text{type}, T.\text{width}); \text{top}(\text{offset}) := \text{top}(\text{offset}) + T.\text{width} \}$
 $D \rightarrow \text{proc id} ; N D ; S \quad \{ p := \text{top}(\text{tableptr}); \text{addwidth}(p, \text{top}(\text{offset})); \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset});$
 $\quad \text{enterproc}(\text{top}(\text{tableptr}), \text{id.name}, \text{nest}, p); \}$
 $N \rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{top}(\text{tableptr})); \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}) ; \text{nest}++; \}$
 $T \rightarrow \text{integer} \quad \{ T.\text{type} := \text{integer}; T.\text{width} := 4 \}$
 $T \rightarrow \text{real} \quad \{ T.\text{type} := \text{real}; T.\text{width} := 8 \}$
 $T \rightarrow \text{id} [\text{num}] \text{ of } T_1 \quad \{ T.\text{width} := \text{num.val} * T_1.\text{width}; T.\text{type} := \text{array}(\text{num.val}, T_1.\text{type}) \}$
 $T \rightarrow \uparrow T_1 \quad \{ T.\text{type} := \text{pointer}(T_1.\text{type}); T.\text{width} := 4 \}$

- The translation scheme described is inadequate to handle recursive procedures. The reason being that the name of such a procedure would be entered in its closest enclosing symbol table only after the entire procedure is parsed.
- However a recursive procedure would have a call to itself within its body and this call cannot be translated since the procedure name would not be present in the table.
- A possible remedy is to enter a procedure's name immediately after it has been found. Semantic actions for only the changed rules are shown.

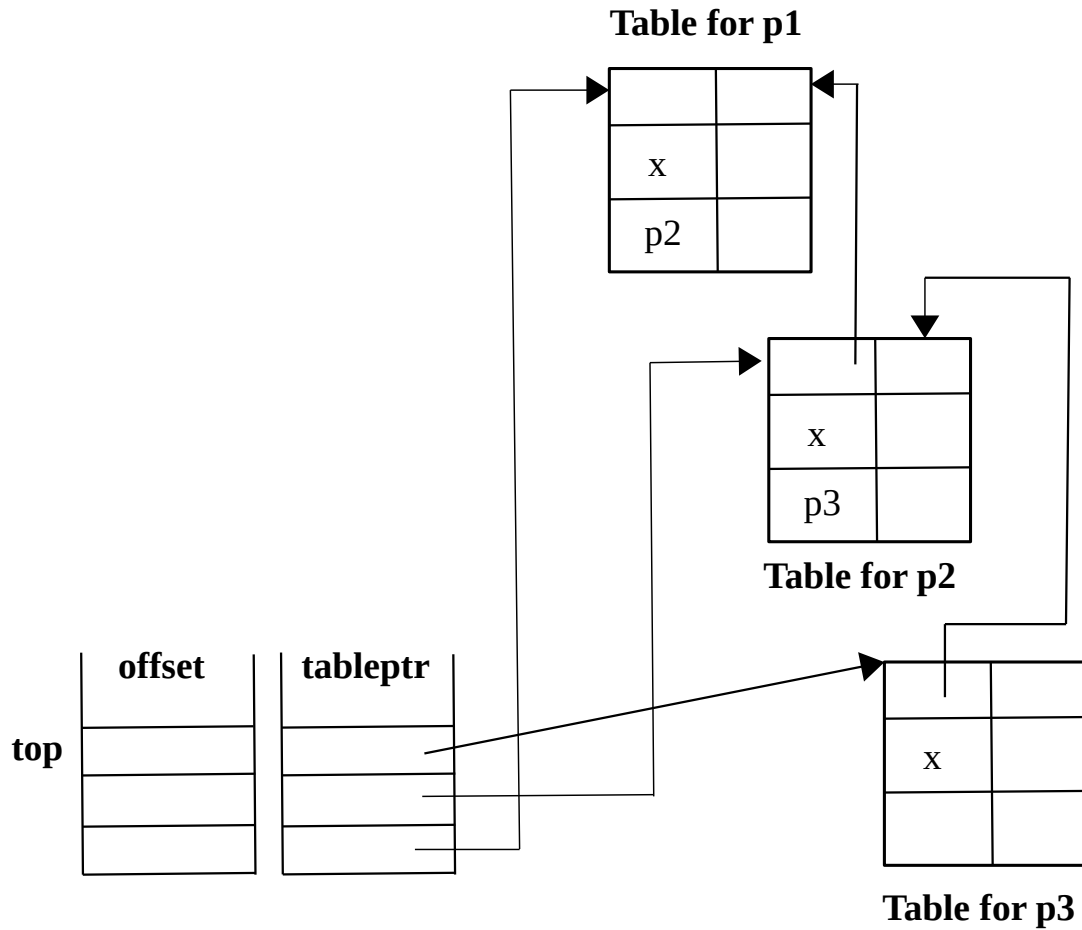
$D \rightarrow \text{proc id} ; N D ; S$
 $\quad \{ p := \text{top}(\text{tableptr}); \text{addwidth}(p, \text{top}(\text{offset})); \text{pop}(\text{tableptr}) ; \text{pop}(\text{offset}); \text{nest}--; \}$
 $N \rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{top}(\text{tableptr})); \text{enterproc}(\text{top}(\text{tableptr}), \text{id.name}, \text{nest}, p)$
 $\quad \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset}) ; \text{nest}++; \}$

Processing of Record Data type

$D \rightarrow D ; D \mid \text{id} : T$
 $T \rightarrow \text{integer} \mid \text{real}$
 $T \rightarrow \text{id} [\text{num}] \text{ of } T_1$
 $T \rightarrow \text{record } L D \text{ end}$
 $L \rightarrow \varepsilon$

Two stacks tableptr and offset are used in the same sense as introduced earlier. Attribute T.width gives the width of all the data objects declared within a record. Attribute T.type is used to hold the type of a record.

$T \rightarrow \text{record } L D \text{ end}$
 $\quad \{ T.\text{type} := \text{record}(\text{top}(\text{tableptr}); T.\text{width} := \text{top}(\text{offset});$
 $\quad \text{pop}(\text{tableptr}); \text{pop}(\text{offset})$
 $\quad \}$
 $L \rightarrow \varepsilon \quad \{ p := \text{mktable}(\text{nil});$
 $\quad \text{push}(p, \text{tableptr}); \text{push}(0, \text{offset})$
 $\quad \}$



(c) Symbol Tables and their interconnection when control reaches the marked point

Sem

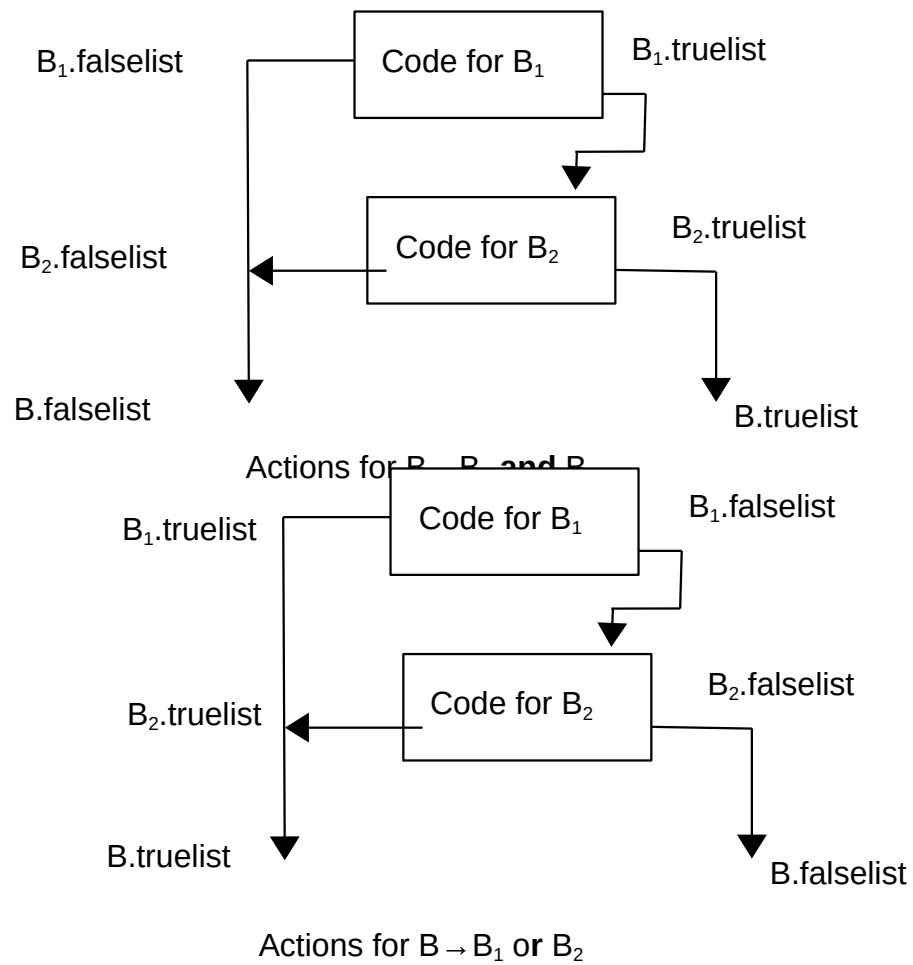
that are detected during processing of declarations of a program include the following among many others

antic

errors

- uniqueness checks
- names referenced but not declared
- names declared but not referenced
- names declared but not initialized properly
- ambiguity in declarations
- and many more

Code for detection and handling of such errors are incorporated in the semantics actions at the relevant places.



Translation of Basic Assignment Statement

```
S → id := E      { p := lookup (id.name); if p ≠ nil then emit ( p ':=' E.place) else error /* undeclared id */ }
E → E1 + E2    { E.place := newtemp(); emit (E.place ':=' E1.place '+' E2.place) }
E → E1 * E2    { E.place := newtemp(); emit (E.place ':=' E1.place '*' E2.place) }
E → - E1       { E.place := newtemp(); emit (E.place ':=' 'uminus' E1.place) }
E → ( E1 )     { E.place := E1.place }
E → id          { p := lookup (id.name); if p ≠ nil then E.place := p else error /* undeclared id */ }
```

Translation of Assignment Statement with Type Conversion

```
E → E1 + E2 { if E1.type = integer and sequiv( E1.type, E2.type)
  then { E.place := newtemp_int(); emit(E.place ':=' E1.place 'int +' E2.place); E.type := integer}
  else if E1.type = real and sequiv( E1.type, E2.type)
    then { E.place := newtemp_real(); emit(E.place ':=' E1.place 'real +' E2.place); E.type := real}
  else if E1.type = real and E2.type = integer
    then { E.place := newtemp_real(); t := newtemp_real(); emit(t ':=' 'inttoreal' E2.place);
      emit( E.place ':=' E1.place 'real +' t); E.type := real }
  else if E1.type = integer and E2.type = real
    then { E.place := newtemp_real(); t := newtemp_real(); emit(t ':=' 'inttoreal' E1.place);
      emit( E.place ':=' t 'real +' E2.place); E.type := real }
  else E.type := type_error
}
S → id := E { p := lookup (id.name) ; if p = nil then error /* undeclared id */
  else if sequiv (type_of(p), E.type) then if E.type = integer
    then { emit ( p 'int:=' E.place) ; S.type := void }
    else { emit ( p 'real:=' E.place); S.type := void }
  else if type_of (p) = real and E.type = integer
    then { t := newtemp_real (); emit ( t ':=' 'inttoreal' E.place); emit ( p 'real:= ' t ) ;
      S.type := void; }
  else S.type := type error; /* type mismatch */ }
```

Note that the temporaries used above are of different types.

Array References (Syntax and semantics of C/C++)

Address Calculations

Address calculation at compile time for an integer array, $a[10,20]$, to find the **address of $a[i_1, i_2]$**

Row major representation with base $a[1, 1]$	Row major representation with base $a[0, 0]$
Let base be the relative address of $a[1,1]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 - 1) * 20 + i_2 - 1) * w$ $= c + v$ where $c = \text{base} - 21 * w$ is a compile time constant and $v = (20 * i_1 + i_2) * w$ is a function of i_1, i_2	Let base be the relative address of $a[0,0]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 * 20 + i_2) * w$ $= c + v$ where $c = \text{base}$ is a compile time constant and $v = (20 * i_1 + i_2) * w$ is a function of i_1, i_2

Similarly we perform address calculations when column-major representation is used.

Column major representation with base $a[1, 1]$	Column major representation with base $a[0, 0]$
Let base be the relative address of $a[1,1]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 - 1) * 10 + i_2 - 1) * w$ $= c + v$ where $c = \text{base} - 11 * w$ is a compile time constant and $v = (10 * i_1 + i_2) * w$ is a function of i_1, i_2	Let base be the relative address of $a[0,0]$. Then the relative address of $a[i_1, i_2]$ is given by $= \text{base} + (i_1 * 10 + i_2) * w$ $= c + v$ where $c = \text{base}$ is a compile time constant and $v = (10 * i_1 + i_2) * w$ is a function of i_1, i_2

Assume an array reference is of the form : $a[e_1, e_2, \dots, e_n]$ in the context of C / C++.

Expression seen by the Parser	Attributes available	Code Generated
$a[e_1,$	$c = \text{addr}(a)$ expression e_1	$t1 \leftarrow \text{evaluate}(e_1)$
$e_2,$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1,$	$t1 := t1 * u_2$ $t2 \leftarrow \text{evaluate}(e_2)$ $t1 := t1 + t2$
$e_3,$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2,$	$t1 := t1 * u_3$ $t2 \leftarrow \text{evaluate}(e_3)$ $t1 := t1 + t2$
$\dots$	$\dots$	$\dots$
$e_{n-1},$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2, e_3, \dots, e_{n-2},$	$t1 := t1 * u_n$ $t2 \leftarrow \text{evaluate}(e_{n-1})$ $t1 := t1 + t2$
$e_n]$	$c = \text{addr}(a)$ $t1 \leftarrow a[e_1, e_2, e_3, \dots, e_{n-1},$	$t2 \leftarrow \text{evaluate}(e_n)$ $t1 := t1 + t2$ $t1 := t1 * w$ $t0 := c$ $t0[t1]$

After processing of $a[e_1, e_2, \dots, e_n]$, this array reference will be replaced by the compiler as $t_0[t_1]$ in its intermediate code in whatever context this array reference occurred in the source.

Note that this scheme attempts to reuse temporaries, as parsing proceeds, and overall only two temporary variables are in use at any given point, eventually saving the result using 2 temporaries, t_0 and t_1 . In view of the translation scheme outlined above, we need to write a grammar that exposes the index expressions one by one so that the parser can

perform syntax and semantic analysis hand in hand.

```

S → L = E { if L.offset = null then emit (L.place := E.place)
             else emit (L.place [' L.offset '] := E.place) }
E → L { if L.offset = null then E.place := L.place
        else E.place := newtemp(); emit (E.place := L.place [' L.offset ']) }
L → id { L.place := id.place; L.offset := null }
L → elist ] { L.place := newtemp(); L.offset := newtemp(); emit(L.place := base(elist.array));
              emit(L.offset := elist.place '*' width(elist.array)) }
elist → elist1 [ E { m := elist1.dim + 1; emit( elist1.place := elist1.place '*' limit(elist1.array, m));
                    emit( E.place := E.place '+' elist1.place );
                    elist.array := elist1.array; elist.place := E.place; elist.dim := m }
elist → id [ E { elist.array := id.place; elist.place := E.place; elist.dim := 1 }

```

Exercise 1 : Formulate the recurrence relations for multidimensional array references permitted in C / C++. Note that the syntax for array reference in C / C++ is different than the form used here.

Exercise 2 : Formulate the recurrence relations for multidimensional array references that are permitted in Fortran like languages that use column major representation with lower bounds of all dimensions starting at 1.

Exercise 3 : Formulate the recurrence relations for multidimensional array references permitted in some languages, which use row major representation but permit negative indices. In general, they allow array declarations of the form :
integer a[L₁:U₁, L₂:U₂, ..., L_n:U_n];

where both L_i and U_i may be negative, but L_i ≤ U_i, 1 ≤ i ≤ n.

In general, given a representation, row-major or column major, the lower and upper bounds, a compiler can always determine how to generate intermediate code along with parsing as each relevant parts of the reference are encountered.

SDTS for Full Evaluation of Boolean expressions is now given.

```

B → B1 or B2 { B.place := newtemp(); emit(B.place := B1.place 'or' B2.place) }
B → B1 and B2 { B.place := newtemp(); emit(B.place := B1.place 'and' B2.place) }
B → not B1 { B.place := newtemp(); emit(B.place := 'not' B1.place) }
B → ( B1 ) { B.place := B1.place }
B → true { B.place := newtemp(); emit (B.place := '1') }
B → false { B.place := newtemp(); emit (B.place := '0') }
B → E { B.place := E.place }
B → E1 relop E2 { B.place := newtemp(); emit ('if' E1.place 'relop.op' E2.place 'goto' nextstat+3);
                  emit (B.place := '0'); emit ('goto' nextstat + 2); emit (B.place := '1')}

```

Scheme2 : Partial evaluation of boolean expressions :

```

B → B1 or M B2 { backpatch (B1.falselist, M.stat); B.truelist := merge (B1.truelist, B2.truelist);
                  B.falselist := B2.falselist }
B → B1 and M B2 { backpatch (B1.truelist, M.stat); B.falselist := merge (B1.falselist, B2.falselist);
                  B.truelist := B2.truelist }
B → not B1 { B.truelist := B1.falselist ; B.falselist := B1.truelist }
B → ( B1 ) { B.truelist := B1.truelist ; B.falselist := B1.falselist }
B → true { B.truelist := makelist (nextstat); emit ('goto ____');
B → false { B.falselist := makelist (nextstat); emit ('goto ____');
B → E1 relop E2 { B.truelist := makelist (nextstat); B.falselist := makelist (nextstat + 1);
                  emit ('if' E1.place 'relop.op' E2.place 'goto ____'); emit ('goto ____') }
B → E { B.place := E.place; B.truelist := makelist (nextstat); B.falselist := makelist (nextstat + 1);
        emit ('if' B.place 'goto ____'); emit ('goto ____') }

```

$M \rightarrow \epsilon \quad \{M.stat := nextstat\}$
 $E \rightarrow E_1 + E_2 \mid \text{other rules for expressions} \quad \{ \text{semantic actions as discussed earlier} \}$

4. SDTS for control flow constructs

$S \rightarrow \text{if } B \text{ then } M S_1$
 $\quad \{ \text{backpatch}(B.truelist, M.stat); S.nextlist := \text{merge}(B.falselist, S_1.nextlist) \}$
 $S \rightarrow \text{if } B \text{ then } M_1 S_1 N \text{ else } M_2 S \quad \{ \text{backpatch}(B.truelist, M_1.stat); \text{backpatch}(B.falselist, M_2.stat);$
 $\quad S.next := \text{merge}(S_1.nextlist, \text{merge}(N.nextlist, S_2.nextlist)) \}$
 $S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1 \quad \{ \text{backpatch}(B.truelist, M_2.stat); \text{backpatch}(S_1.nextlist, M_1.stat);$
 $\quad S.nextlist := B.falselist; \text{emit}('goto' M_1.stat) \}$
 $S \rightarrow \text{begin } S_l \text{ end} \quad \{ S.nextlist := S_l.nextlist \}$
 $S_l \rightarrow S_{l1}; M S \quad \{ \text{backpatch}(S_{l1}.nextlist, M.stat); S_l.nextlist := S.nextlist \}$
 $S_l \rightarrow S \quad \{ S_l.nextlist := S.nextlist \}$
 $S \rightarrow L := E \quad \{ \text{semantic actions as earlier for assignment}; S.nextlist := nil \}$
 $M \rightarrow \epsilon \quad \{ M.stat := nextstat \}$
 $N \rightarrow \epsilon \quad \{ N.nextlist := \text{makelist}(nextstat); \text{emit}('goto ___') \}$

5. Translation of Other constructs : The basic issues in semantic analysis and translation have been covered. However, a compiler has to deal with more details, a few of which are listed below. Translation of goto statement or break statements. It is handled by storing labels in the symbol table, associating index of 3-address code with the label and using backpatching to resolve forward references.

Procedure call statement

Consider a function definition :

```

void func (int a, int * p, float f)
{ /* body skipped */ return; }

```

and a corresponding call to func() as given below.

```

func(i*j+5, &q+5, 25 *pi); // i, j are int; q is int*; and pi is float)

```

the desired intermediate code is. Note how each actual argument is evaluated into a temporary of the same type as that defined in the corresponding formal argument. The use of param statements, one for each actual parameter followed by a call to the function.

10: t1 := i * j	17: t8 := intoreal t6
11: t2 := t1 + 5	18: t9 := t8*pi
12: t3 := &q	19: param t2
13: t4 := 5*4	19 : param t6
14: t5 := t3+t4	20 : param t9
15: t6 := &t5	21 : call func
16 : t7 = 25	

Parameter passing may be call by value or call by reference and are processed accordingly..

- Semantic analysis involves type checking of the formal and actual arguments.
- Translation would require generating 3-address code for evaluating each argument (in case it is an expression). Then generate a list of 3-address **param** statements, one for each argument, followed by the 3-address call statement.

Translation of other constructs such as **case** statement, can be worked out on similar lines.